

Perfeito. Nesse caso, eu mudaria a conversa com o desenvolvedor: **v**. Como seu aplicativo tem vários parceiros vendidos dentro da plataforma, o requisito técnico central é uma **infraestrutura de pagamentos para marketplace**, com **divisão de pagamento**, **vendedores múltiplos**, **recorrência**, **Pix**, **cartões**, **boleto**, **carteiras digitais**, e .

E um ponto importante: **não existe uma lista finita de “todos os gateways existentes”** . Existem centenas de PSPs, adquirentes, gateways e orquestradores no mundo. O que faz sentido é especificar uma **arquitetura que permite conectar vários provedores**, sem ficar preso em um único gateway.

Abaixo está um documento que você pode enviar literalmente para o desenvolvedor.

ESPECIFICAÇÃO TÉCNICA — INFRAESTRUTURA DE PAGAMENTOS DO APP

1. OBJETIVO

O aplicativo deve operar como **marketplace/plataforma de benefícios**, permitindo que múltiplos parceiros (vendedores) comercializem produtos, serviços, vouchers e experiências dentro do aplicativo.

A infraestrutura deverá permitir:

- cadastro de parceiros/vendedores;
- finalizar compra dentro do aplicativo;
- meios múltiplos de pagamento;
- divisão automática de pagamentos;
- comissão da plataforma;
- repasse automático aos parceiros;
- pagamentos recorrentes;
- estornos;
- estornos;
- conciliação financeira;
- webhooks;
- antifraude;
- tokenização de ;
- múltiplos gateways/PSPs;
- possibilidade de adicionar novos gateways no futuro sem reescrever o checkout.

Requisito arquitetônico principal: PAYMENT ORCHESTRATION LAYER.

2. MEIOS DE PAGAMENTO QUE O APP DEVERÁ SUPORTAR

PIX

Implementar:

- Código QR dinâmico;
- Pix Copia e Cola;
- confirmação automática;
- webhook de confirmação;
- expiração da cobrança;
- estorno/devolução;
- conciliação;
- identificação da transação;
- Pix recorrente/Pix Automático quando suportado pelo PSP;
- divisão de Pix quando suportado.

O Pix deve ser tratado como **pagamento assíncrono**, e o backend não deve considerar uma venda concluída simplesmente porque o frontend exibe o QR Code.

CARTÃO DE CRÉDITO

r:

- crédito à vista;
 - parcelamento;
 - parcelamento com juros;
 - parcelamento sem juros;
 - captura automática;
 - pré-autorização + captura;
 - tokenização;
 - salva;
 - compra com um clique;
 - recorrência;
 - estorno total;
 - estorno parcial;
 - estorno;
 - 3DS/3DS2 quando disponível.
-

CARTÃO DE DÉBITO

Suportar quando disponível pelo PSP/adquirente escolhido.

BOLETO

r:

- boleto registrado;
 - código de serviço;
 - linha digitada;
 - ✓;
 - confirmação automática;
 - leal;
 - ;
 - conciliação;
 - estorno quando aplicável.
-

CARTEIRAS DIGITAIS

Prever suporte para:

- Apple Pay;
- Google Pay;
- Samsung Pay;
- PayPal;
- outras carteiras oferecidas pelo PSP.

A disponibilidade varia conforme o provedor e o contrato. Por exemplo, a documentação da Cielo Apple Pay, Google Pay, Samsung Pay e PayPal em determinados cenários de integração.  Cielo D...

3. RECORRÊNCIA

O sistema deverá:

Assinaturas

- mensal;
- trimestral;
- semestral;
- anual;
- período de teste;

- renovação automática;
- leal;
- pausa;
- retomada;
- atualizar;
- rebaixar;
- cobrança proporcional;
- tentar novamente;
- recuperação de pagamentos recusados.

Formas de recorrência

- ;
- Pix Automático, quando disponível;
- outros meios apoiados pela PSP.

4. PAGAMENTO DIVIDIDO — REQUISITO CRÍTICO

Esse é provavelmente o ponto **mais importante de todo o projeto** .

Imaginar:

Cliente compra R\$ 100

Regra:

- Parceiro: R\$ 85
- Plataforma: R\$ 10
- outro participante: R\$ 5

O sistema deverá executar a divisão automaticamente.

De preferência:

Cliente → PSP → distribuição automática

e não:

Cliente → sua empresa → você transfere manualmente para parceiro.

Mercado Pago, Pagar.me, iugu, PagBank, Cielo e Asaas, por exemplo, possuem soluções documentadas de split/marketplace, embora as regras e responsabilidades sejam diferentes entre eles.  Mercado Pago +5

5. O SISTEMA DEVE PERMITIR SPLIT POR:

Percentual

Exemplo:

Parceiro 90% / Plataforma 10%

Valor

Exemplo:

Parceiro recebe R\$ 80

Plataforma recebe R\$ 20

Múltiplos recebedores

Exemplo:

Parceiro A — 70%

Parceiro B — 20%

Plataforma — 10%

Regras diferentes por parceiro

Exemplo:

Restaurante:

90/10

Moda:

85/15

Experiência:

80/20

6. RESPONSABILIDADE POR TAXAS E ESTORNO

O sistema precisa permitir configurar:

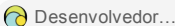
Quem paga a taxa do gateway?

Quem escaneia o chargeback?

Quem absorveu o custo do estorno?

Isso não deve ser codificado de forma fixa.

Deve ser uma **regra configurável pelo vendedor/contratado** .

Essa preocupação é importante porque os PSPs tratam responsabilidades de forma diferente. No PagBank, por exemplo, a documentação de split informa que o recebedor primário pode ficar responsável pelas taxas e pelo estorno. 

7. CADASTRO DOS PARCEIROS — SELLER ONBOARDING

Cada parceiro deverá possuir um cadastro próprio.

Dados:

- razão social;
- nome fantasia;
- CNPJ/CPF;
- endereço;
- legal
- e-mail;
- telefone;
- dados bancários;
- dados fiscais;
- conta/recebedor no PSP;
- status KYC;
- status de aprovação;
- percentual de comissão;
- regras de repasse.

O sistema deverá permitir:

PENDENTE → EM ANÁLISE → APROVADO → ATIVO → SUSPENSO → BLOQUEADO

O Pagar.me, por exemplo, utiliza uma estrutura `recipient` para representar os vendedores/empresas que participam do marketplace. 

8. FINALIZAR A COMPRA

O checkout deverá ser **modular** .

O cliente deverá conseguir escolher:

PIX

CARTÃO

BOLETO

CARTEIRA

E o sistema deverá mostrar apenas os meios disponíveis para transação.

9. ORQUESTRADOR DE PAGAMENTOS

Eu pediria ao desenvolvedor específico:

Não integrar o aplicativo diretamente a um único gateway. Crie uma camada de abstração/orquestração de pagamentos.

Exemplo:

```
APP
  ↓
PAYMENT SERVICE
  ↓
PAYMENT ORCHESTRATOR
  ↓
├─ Mercado Pago
├─ Pagar.me
├─ PagBank
├─ Asaas
├─ iugu
├─ Stripe
├─ Adyen
└─ outros PSPs
```

Assim, se amanhã você trocar Pagar.me por outro fornecedor, **o aplicativo não precisa ser reconstruído** .

10. GATEWAYS / PSPs QUE EU DEIXARIA PREVISTOS

PRIORIDADE BRASIL

Pagar.me / Pedra

Muito interessante para marketplace e split. A API atual documenta pedidos com split e múltiplos recebedores.  Pagar... +1

Mercado Pago

Possui checkout transparente e Checkout Pro com split para marketplace.  Mercado P...

PagBank / PagSeguro

Possui opção de pagamento para cartão, boleto e Pix, inclusive com múltiplos recebedores.  Desenvolvedor...

iugu

Possui split, multisplit, subcontas, recorrência e diferentes meios de pagamento.  iugu

Asaas

Possui Pix, cartão, boleto, recorrência e parcelamento entre contas.  Asaas - Docum... +1

Céu / Braspag

Possui solução de split para marketplaces e diferentes meios de pagamento.  Céu

11. OUTROS PSPs/GATEWAYS A DEIXAR COMO ADAPTERS FUTUROS

Brasil

- Pedra
- Getnet
- Rede
- SafraPay
- Vindi
- Zoop
- Doca
- Celcoin
- EBANX
- Appmax
- PayPal

- Mercado Pago
- PagBank
- Pagar.me
- Asaas
- iugu
- Céu/Braspag

internacionais

- Listra
- Adyen
- PayPal/Braintree
- Checkout.com
- Worldpay
- Nuvei
- Rapyd
- dLocal
- Airwallex

Não significa que todos precisam ser integrados no lançamento.

Significa que a arquitetura deve permitir adicioná-los posteriormente.

12. O SISTEMA DEVE TER UM “ADAPTADOR DE PAGAMENTO”

Conceitos de exemplo:

```
PaymentProvider
|
├─ createPayment()
├─ authorizePayment()
├─ capturePayment()
├─ cancelPayment()
├─ refundPayment()
├─ getPaymentStatus()
├─ createCustomer()
├─ tokenizeCard()
├─ createSubscription()
├─ cancelSubscription()
├─ createSplit()
├─ createSeller()
└─ handleWebhook()
```

Cada gateway implementa seu próprio adaptador.

Assim:

```
PagarmeAdapter  
MercadoPagoAdapter  
PagBankAdapter  
AsaasAdapter  
IuguAdapter  
StripeAdapter  
AdyenAdapter
```

O restante do aplicativo conversa apenas com:

Serviço de Pagamento

e não diretamente com o gateway.

13. WEBHOOKS

Obrigatório.

O backend deverá receber eventos como:

```
payment.created  
payment.pending  
payment.authorized  
payment.paid  
payment.failed  
payment.expired  
payment.refunded  
payment.partially_refunded  
payment.chargeback  
payment.disputed  
  
subscription.created  
subscription.renewed  
subscription.failed  
subscription.cancelled  
  
seller.created  
seller.approved  
seller.rejected  
seller.suspended
```

O sistema TM ter:

- idempotência;
- tentar novamente;

- assinatura/verificação do webhook;
 - registros;
 - fila de;
 - tratamento de eventos duplicados.
-

14. CONCILIAÇÃO FINANCEIRA

O administrador deverá conseguir visualizar:

POR VENDA

- valor bruto;
- desconto;
- taxas do gateway;
- comissão da plataforma;
- valor do parceiro;
- valor;
- status;
- método de pagamento;
- data de pagamento;
- data de ajuste.

POR PARCEIRO

Vendas
Cancelamentos
Estornos
Chargebacks
Comissões
Taxas
Valor líquido
Saldo
A receber
Já recebido

15. PAINEL DO PARCEIRO

Cada parceiro deverá ter um dashboard.

“MINHAS VENDAS”

- hoje;
- 7 dias;

- 30 dias;
- mês;
- histórico.

“MEU SALDO”

- disponível;
- .
- receber;
- ...

“MINHAS TRANSAÇÕES”

Com filtro por:

- Pix;
- ;
- boleto;
- período;
- status.

16. CUPOM / VALE

Como seu aplicativo será um clube de benefícios, eu peço ao desenvolvedor para não limitar o sistema de produtos físicos.

existir:

PRODUTO

SERVIÇO

EXPERIÊNCIA

VALOR

CUPOM

BILHETE

SUBSCRIÇÃO

Isso vem do vendedor:

 R\$ 200 por R\$ 149

e gerar automaticamente:

ID_DO_COMPROVANTE

CÓDIGO QR

TOKEN_ÚNICO

para o parceiro validar.

17. ESTORNO

O backend deve permitir:

Estorno integral

R\$ 100 → R\$ 100

Estorno parcial

R\$ 100 → R\$ 40

Cancelamento antes da captura

Quando o PSP permitir.

E o sistema deve recalcular:

comissão + divisão + saldo do parceiro.

18. SEGURANÇA

Não :

- número completo do caso;
- CVV;
- dados sensíveis de autenticação.

Usar:

tokenização / campos hospedados / SDK oficial do PSP.

Além disso:

- PCI DSS;
- LGPD;

- ;
 - segredos exclusivamente sem back-end;
 - Chaves de API fora do frontend;
 - que forte;
 - registros;
 - antifraude;
 - limitação de taxa;
 - chaves de idempotência.
-

19. ANTIFRAUDE

Pré-integração com:

- Radar/antifraude para PSP;
 - 3DS2;
 - análise de risco;
 - Impressão digital do dispositivo;
 - verificações de velocidade;
 - lista negra;
 - lista branca;
 - regras por valor;
 - regras por CPF;
 - regras por cada;
 - regras por IP/dispositivo.
-

20. O QUE EU COLOCARIA NO MVP

Para não transformar o projeto em um Frankenstein de 47 APIs 😄:

FASE 1

PIX

Cartão de crédito

Boleto

Recorrência

Dividir

Estorno

Webhooks

KYC dos parceiros

Painel de controle

Voucher/Código QR

Antifraude

PSPs iniciais:

Pagar.me + Mercado Pago + Asaas

ou, dependendo das condições comerciais que você conseguir:

Pagar.me + Mercado Pago + PagBank.

O importante é que o desenvolvedor construa uma **camada de abstração**, e não três integrações espalhadas pelo código.

E UMA DECISÃO QUE EU TOMARIA AGORA

Para o seu modelo, eu **não venderia a ideia ao desenvolvedor como “quero vários gateways”**.

Eu entregaria exatamente isto:

“Quero uma Camada de Orquestração de Pagamento para um marketplace de benefícios, com arquitetura multi-PSP, seller onboarding, split payment, recorrência, Pix, cartão, boleto, wallets, antifraude, webhooks, estorno, chargeback e conciliação. O gateway deve ser intercambiável por adaptador.”

Essa frase já muda completamente o nível técnico do projeto.

E o **split é realmente central** para o seu caso: Pagar.me, por exemplo, permite definir recebedores e percentuais/valores no pedido; Mercado Pago faz divisão entre marketplace e vendedor; iugu permite multisplit inclusivo; e Asaas permite dividir entre diferentes contas.  Pagar.... +3

Eu também peço ao desenvolvedor para não começar a codificar antes de definir quem será o “Comerciante de Registro”/responsável pela cobrança, quem será o recebedor principal e como serão tratados estornos, reembolsos e repasses. Isso

impacta diretamente o desenho jurídico, financeiro e técnico do mercado — não é apenas uma decisão de programação.